

P02 ❖❖❖ Occurrence Semantics and the Concurrent React Adapter

Executable research capsule

PBUI research prototype

2026-08-05

Contents

1	P02 - Occurrence Semantics and the Concurrent React Adapter	1
1.1	Research question	1
1.2	Subject versus occurrence	2
1.3	Lifecycle protocol	2
1.3.1	Prepare	2
1.3.2	Commit	2
1.3.3	Update	3
1.3.4	Cleanup	3
1.4	Why generations are necessary	3
1.5	Activation evidence	3
1.6	Surface and reachability semantics	4
1.7	Renderer-neutral commit bridge	4
1.8	Formal model	5
1.9	Tests and experiments	5
1.10	Results established by the prototype	6
1.10.1	By construction	6
1.10.2	By executable tests	6
1.10.3	Not yet proved	6
1.11	Deliberate limitations	6
1.12	Integration seam	6
1.13	Research extensions	7

1 P02 - Occurrence Semantics and the Concurrent React Adapter

1.1 Research question

How should a presentation runtime represent the lifetime of a rendered semantic occurrence when a modern renderer may prepare, abandon, commit, update, remount, hide, virtualize, and clean up UI work out of the simple one-render/one-unmount order assumed by imperative registration APIs?

This capsule builds a renderer-neutral commit-phase occurrence registry and tests it against the lifecycle hazards associated with concurrent and strict rendering.

1.2 Subject versus occurrence

A semantic subject is something such as:

```
{ sort: "field", key: "doc-readings/temperature" }
```

An occurrence is one addressable presentation of that subject:

```
{
  id: "occ-chart-temperature",
  subject: { sort: "field", key: "doc-readings/temperature" },
  forms: ["field-chip"],
  surface: "workspace",
  placement: "placement-chart",
  visible: true,
  reachable: true,
  enabled: true,
}
```

The same subject may have several occurrences: a chart label, table header, source-browser chip, and command-history entry. Removing one occurrence does not remove the subject.

This distinction is required for PBUI input contexts. A subject may be a valid semantic candidate but have no currently mounted pointer-selectable occurrence. Conversely, one subject may be selectable through several visual forms.

1.3 Lifecycle protocol

1.3.1 Prepare

```
const prepared = registry.prepareMount(descriptor);
```

Preparation is pure with respect to the committed registry. It can happen during a speculative renderer phase. If the render is abandoned, `abortPrepared` leaves no semantic occurrence behind.

1.3.2 Commit

```
const lease = registry.commitMount(prepared);
```

Commit publishes one occurrence and returns a generation lease:

```
interface OccurrenceLease {
  occurrenceId: string;
  generation: number;
  leaseId: string;
}
```

Committing the same prepared token twice is idempotent.

1.3.3 Update

```
const preparedUpdate = registry.prepareUpdate(lease, nextDescriptor);
const nextLease = registry.commitUpdate(preparedUpdate);
```

An update is atomic. Before commit, readers observe the old descriptor. After commit, they observe the new descriptor and a greater generation. Subject, forms, visibility, reachability, and metadata change as one committed revision.

1.3.4 Cleanup

```
registry.cleanup(lease);
```

Cleanup succeeds only when the lease still names the active generation. An old cleanup cannot delete a newer remount or update.

This is the central stale-cleanup law:

```
activeGeneration(id) != lease.generation
=> cleanup(lease) is observationally inert
```

1.4 Why generations are necessary

Consider this legal interleaving:

```
commit occurrence X generation 1
prepare and commit replacement X generation 2
late cleanup for generation 1
```

A registry keyed only by occurrence ID would delete generation 2. The lease makes cleanup conditional on both ID and generation.

The same mechanism protects activation evidence. A click captured against generation 1 is invalid after generation 2 changes the subject or reachability.

1.5 Activation evidence

```
interface ActivationEvidence {
  occurrenceId: string;
  generation: number;
  subject: SubjectRef;
  surface: string;
  registryRevision: number;
}
```

activationEvidence(id) returns evidence only for an occurrence that is:

- committed;

- visible;
- reachable;
- enabled.

validateEvidence checks that the occurrence still has the same generation, subject, and surface and remains activatable.

This prevents a delayed pointer event from selecting a different subject after a keyed component has been repurposed.

1.6 Surface and reachability semantics

The registry records separate facts:

- **mounted**: implied by presence in the registry;
- **visible**: presently rendered rather than intentionally hidden;
- **reachable**: available through the current interaction modality;
- **enabled**: not locally disabled;
- **surface**: workspace, menu, overlay, command palette, or another semantic surface.

These should not be collapsed into one Boolean. A screen-reader surface, pointer surface, and searchable semantic subject set can differ.

selectable supports filters but does not itself decide an input-context query. P03 and P04 should derive semantic applicability; P02 supplies committed occurrence facts.

1.7 Renderer-neutral commit bridge

The capsule includes createCommitPhaseBridge. It models the portion of a React hook that must remain stable across renders:

```
const bridge = createCommitPhaseBridge(registry);
const prepared = bridge.prepare(descriptor); // render-safe
bridge.commitPrepared(prepared);           // commit phase
bridge.cleanup();                          // effect cleanup
```

A concrete React hook can be built around useRef and useEffect. The important constraint is that render must not publish an occurrence.

Illustrative adapter:

```
function useSemanticOccurrence(descriptor: OccurrenceDescriptor) {
  const runtime = useRuntime();
  const bridgeRef = useRef<CommitPhaseBridge>();
  bridgeRef.current ??= createCommitPhaseBridge(runtime.occurrences);

  const prepared = bridgeRef.current.prepare(descriptor);

  useEffect(() => {
    bridgeRef.current?.commitPrepared(prepared);
    return () => {
      bridgeRef.current?.cleanup();
    };
  });
}
```

```
    }, [prepared]);
  }
```

A production hook needs careful memoization so it does not treat every render as a semantic update, and it must align with the chosen React version’s external-store recommendations. The capsule isolates the semantic protocol that such a hook must implement.

1.8 Formal model

Let an active occurrence map be:

$$A : \text{OccurrenceId} \rightarrow (\text{Generation} \times \text{Descriptor}).$$

A lease is valid in state A when:

$$\text{valid}_A(i, g, \ell) \iff A(i) = (g, d) \wedge d.\text{leaseId} = \ell.$$

Cleanup is:

$$\text{cleanup}(A, L) = \begin{cases} A \setminus \{L.i\}, & \text{valid}_A(L), \\ A, & \text{otherwise.} \end{cases}$$

Commit replaces the current record with a record at a strictly greater generation. Prepared values do not occur in A, so abandoning one is unobservable.

An activation certificate is valid only if its generation and semantic coordinates agree with the active record.

1.9 Tests and experiments

The eleven executable tests cover:

1. speculative preparation with no committed side effect;
2. idempotent commit of one prepared mount;
3. stale cleanup after a newer remount;
4. atomic subject and visibility update;
5. rejection of an update using an old lease;
6. several occurrences presenting one subject;
7. surface, visibility, reachability, and enabled filtering;
8. activation evidence invalidation after update and unmount;
9. virtualization removing an occurrence but not its subject coordinate;
10. renderer-neutral prepare/commit/update/cleanup bridge behavior;
11. coherent subscription snapshots after mutations.

Run:

```
npm test
npm run demo
```

1.10 Results established by the prototype

1.10.1 By construction

- Speculative preparation does not alter committed occurrence state.
- Every committed mount or update receives a monotonically increasing generation for its occurrence ID.
- Cleanup and update require a matching lease.
- Occurrence IDs and semantic subject references are separate types and fields.

1.10.2 By executable tests

- Strict/remount-style late cleanup cannot remove the newest generation.
- Activation evidence becomes invalid after a semantic update.
- Multiple occurrences of one subject remain independently addressable.
- Subscribers see snapshots at coherent registry revisions.

1.10.3 Not yet proved

- linearizability under actual multithreaded mutation;
- behavior under React Offscreen and future renderer semantics;
- absence of tearing in a concrete useSyncExternalStore adapter;
- liveness of keyboard focus transfer during virtualization;
- refinement between DOM event traces and the semantic event protocol.

1.11 Deliberate limitations

1. The prototype is renderer-neutral and does not import React.
2. It uses one active record per OccurrenceId; callers must allocate distinct IDs for simultaneous visual copies.
3. It does not derive visibility from the DOM or intersection observers.
4. It does not persist occurrence state; occurrences are transient by definition.
5. It uses exact (sort, key) subject comparison. A future adapter should delegate alias-aware comparison to P01.
6. It does not resolve modality precedence when several input contexts are active.
7. It does not decide actions or selection predicates.

1.12 Integration seam

A future composed runtime should publish P02 records as extensional facts:

```
presents(occurrenceId, subjectSort, subjectKey)
form(occurrenceId, formName)
surface(occurrenceId, surfaceName)
visible(occurrenceId)
reachable(occurrenceId)
enabled(occurrenceId)
generation(occurrenceId, n)
```

P03 can query those records directly. P04 can derive selectable and availableAction. Before commit, the orchestrator validates P02 activation evidence and P03/P04 evidence

together.

1.13 Research extensions

- implement and test a real React useSyncExternalStore adapter;
- model focus, hover, pointer capture, and keyboard modality as separate occurrence relations;
- add visibility-observer and virtualization adapters;
- specify occurrence traces as a labeled transition system;
- compare generation leases with epoch-based and capability-based lifecycle models;
- model server-rendered and hydrated occurrences;
- build a TLA+ specification for stale cleanup and double-resolution races;
- prove trace refinement between a React test renderer and the pure occurrence machine.